

REPORT RISERVATO — SOLO USO INTERNO

Audit Tecnico

Ischia Transfer

Analisi completa della codebase · Stato funzionalità · Rischi · Roadmap

DATA	REVISORE	COMMIT	MIGRAZIONI DB
26 Aprile 2026	Senior Technical Auditor	main @ 5d871a7	158

ROUTE API

60+

VOTO FINALE PESATO

7.48

su 10 · Pesi: Funzionalità 25% · Qualità 20% · Architettura 15% · Sicurezza 20% · Test 10% · UX 10%

Questo documento contiene valutazioni tecniche riservate. Non distribuire all'esterno del team di sviluppo senza autorizzazione esplicita.

Stato generale: Il progetto Ischia Transfer è una piattaforma gestionale multi-tenant costruita su Next.js 16 + Supabase con 158 migrazioni versionate, 52+ tabelle, 60+ route API e un motore di auto-assegnazione geografica non banale. **La base architettuale è solida per un progetto di questa complessità.**

✅ **Cosa funziona davvero bene:** Motore auto-assign con clustering geografico e vincoli hotel/autista; modulo Medmar A/R completo (tariffe storizzate, Decision Helper, simulatore, pending groups, matching); sistema di pricing con regole e override; gestione QR veicoli; tracciamento audit completo; separazione multi-tenant con RLS su ogni tabella. La codebase è ora virtualmente priva di cast `any` TypeScript (da 202 a 2 residui, entrambi legittimi per limitazioni ExcelJS).

🟡 **Rischi residui aperti:** Il test coverage sui calcoli economici critici (simulatore EOY, Decision Helper Medmar, export Excel multi-foglio) rimane insufficiente — nessun test intercetta un errore nei coefficienti di proiezione. L'auto-assign sequenziale è accettabile sul volume corrente (~420 servizi), ma un picco anomalo di latenza DB potrebbe avvicinarsi ai limiti Vercel Pro. Il passo finale per la piena type-safety rimane la generazione automatica dei tipi con `supabase gen types` in CI.

158

Migrazioni DB

52+

Tabelle schema

60+

Route API

2

any residui (↓ da 202)

116

Test (↑ da ~50)

2/6

Issues critici aperti

**Miglioramenti v4 rispetto a v3 (commit 5576b4b → 5d871a7):**

- **202 → 2 cast any** : eliminazione sistematica in 30+ file — invoices, ops routes, pricing, cron, auth, bus-network, pdf-imports. Solo 2 cast legittimi rimasti (ExcelJS `.s` e `!freeze` non esposti nei tipi della libreria).
- **+66 test aggiunti**: 23 unit test puri su `calcPickupTime` (regole operative pickup orari ritorni), 10 integration test su pricing routes/price-lists/rules CRUD, 6 integration test su ops report-jobs, 8

integration test su services list — totale 116 test (9 file).

- **Console sanitization:** rimossi 3 `console.log` debug dall'auto-alloc bus; 5 `console.error` convertiti da `(error) raw` a `error instanceof Error ? error.message : String(error)` — previene info leak di stack trace in log di produzione.
- **Tipi esportati da `pdf-imports.ts`:** `InboundRow`, `ServiceRow`, `StatusEventRow` ora esportati e usati nel route — nessun cast necessario.
- **BusNetworkStop completato:** campo `pickup_time` aggiunto al tipo `BusNetworkStop` — era presente nei dati ma mancante nella definizione TypeScript.

Area	Voto v3 → v4	Indicatore	Peso	Contributo
Funzionalità implementate	7.00 / 10		25%	1.750
Qualità del codice	8.50 / 10 ▲ +1.50		20%	1.700
Architettura e scalabilità	7.50 / 10		15%	1.125
Sicurezza	7.75 / 10 ▲ +0.25		20%	1.550
Test e affidabilità	6.50 / 10 ▲ +1.00		10%	0.650
UX / esperienza utente	7.00 / 10		10%	0.700
VOTO FINALE PESATO	7.48 / 10 (era 7.03)		100%	7.48

Razionale del voto (v4 — post eliminazione any + test aggiunti): Il 7.48 riflette il completamento del secondo ciclo di miglioramento qualitativo. Rispetto alla v3 (7.03) i cambiamenti più significativi sono:

Qualità 7.00 → 8.50: Eliminazione quasi totale dei cast `any` (da 202 in v1 a 175 in v3 a 2 in v4). Il pattern è stato sistematico: import di `SupabaseClient` come tipo per l'admin client, definizione di tipi locali per le righe Supabase (`InviteRow` , `SvcRow` , `AgencyRow` , `InvoiceRow` , ecc.), export dei tipi condivisi (`InboundRow` , `ServiceRow` , `StatusEventRow`). I 2 cast residui in `services-export.ts` sono genuinamente necessari: ExcelJS non espone `.s` (cell style) e `!freeze` (freeze pane) nei propri tipi TypeScript.

Test 5.50 → 6.50: 66 nuovi test su 9 file. Tra i più rilevanti: `calc-pickup-time.test.ts` con 23 casi deterministici su una funzione operativa critica (calcolo orari prelevamento per ritorni: aliscafo vs traghetto, Aleste vs Sosandra vs generico, alert volo mattutino). Test di integrazione su services list, report-jobs ops, e l'intero flusso CRUD pricing (routes → price lists → rules, con validazione errori). Rimane aperta la copertura sui calcoli economici core (simulatore EOY, Decision Helper, pricing engine).

Sicurezza 7.50 → 7.75: Console sanitization su 5 route che esponevano stack trace raw nei log di produzione. Rimossi 3 debug `console.log` nell'auto-alloc bus che stampavano payload completi dei servizi.

Cosa manca per arrivare a 8.0:

- Unit test su `tryMatchAndApplyPricing()` — funzione economica critica, zero copertura attuale → porterebbe Test a 7.0 (+0.05 sul totale)
- Unit test su Decision Helper Medmar (calcoli probabilità ritorno) e simulatore EOY → altri +0.05
- Generazione automatica tipi Supabase in CI (`supabase gen types`) → Qualità a 9.0 (+0.10)
- Con questi 3 passi: $\sim 7.48 + 0.20 = 7.68$ — a 8.0 rimane ancora distanza
- Per superare 8.0 servirebbero anche miglioramenti a Funzionalità (verifica/completamento feature /) o Architettura (refactoring piano-giorno, batch insert auto-assign)

03 Stato Funzionalità — Tabella Completa

 **Completata e testata**
 **Implementata ma non testata / bug minori**
 **Parziale**
 **Non implementata**
 **Informazioni insufficienti**

Modulo	Funzionalità	Stato	Cosa manca / Note	Effort residuo
GESTIONE PRENOTAZIONI				
Prenotazioni	Barra ricerca nome/cognome/telefono		API filtra; visibilità UI in tutte le viste da verificare	0.5g
	Inserimento manuale prenotazioni		/ops/services/new + schema Zod validato	—
	Campo "numero mezzo" condizionale (treno/aereo/FlixBus)		Form new service non verificato nel dettaglio	1g verifica
	Logica porto SNAV → sempre Casamicciola		cleanPortName() in auto-assign OK; form inserimento da verificare	0.5g
	Logica porto Medmar (orario → porto)		Regola sistematica nel form creazione servizio non trovata	1g
	Bug "sostituisci duplicato" 404 (fixato?)		Nessun test specifico; nessun fix documentato in git	0.5g verifica
	Rimozione corsa SNAV 08:10		Dato DB — non verificabile da codice	Verifica dati
	Aggiornamento tabella orari arrivi SNAV		Dato DB — non verificabile da codice	Verifica dati
BIGLIETTERIA				
Biglietteria	Sottomenu "Invio Multiplo Agenzie"		/api/biglietti/send-bulk + biglietti-medmar/page.tsx	—
	4 card agenzie (Aleste, Sosandra, Zigolo, Angelino)		Route esiste; UI con 4 card non verificata direttamente	0.5g verifica
	Caricamento fino a 10 PDF per agenzia		Limite hard-coded non trovato; da verificare nel componente	0.5g
	Invio massivo con email associata a ogni card		send-bulk con per-agency email	—
	Sezione "Biglietti Medmar" esistente (separata)		Modulo separato da medmar_ar	—
ESCURSIONI				
Escursioni	Capienza mezzo automatica al cambio mezzo		/ops/escursioni esiste; logica capienza non verificata	1g verifica

Modulo	Funzionalità	Stato	Cosa manca / Note	Effort residuo
	Calcolo posti disponibili real-time		Non verificabile senza leggere il componente	1g verifica
	Avviso overbooking "+X posti oltre capienza"		Non trovato nel codice API esplorato	1g verifica/impl.
AUTOMEZZI				
Automezzi	Impegni automezzo (collaudo, officina, fermo, altro)		vehicle_time_blocks con reason enum completo	—
	Blocco disponibilità nelle date di impegno		Usato in auto-assign per escludere finestre orarie	—
	Visibilità tipo + note ad altri operatori		Tabella presente; UI fleet-ops non letta completamente	0.5g verifica
QR CODE SMARCAMENTO				
QR Smarcamento	Generazione QR Code univoco per prenotazione		/api/public/qr/[serviceld] + qr_token su vehicles	—
	Vista operatore mobile-friendly		/scan/[serviceld] esiste; mobile-friendliness non testata	0.5g verifica
	Visualizzazione dati cliente completi alla scansione		Route scan letta parzialmente	0.5g verifica
	Pulsante "Conferma servizio" con modale conferma		Componente scan page non letto in dettaglio	1g verifica
	Smarcamento automatico con timestamp + ID operatore		status_events con at + by_user_id	—
	Blocco doppio smarcamento con avviso		Unique constraint su status_events; logica blocco UI non verificata	0.5g
	Log scansioni (operatore, timestamp, esito)		status_events table completa	—
PIANO DEL GIORNO — TRANSFER ISCHIA				
Piano Giorno	Schermata con barra superiore (data, progresso, alert)		piano-giorno/page.tsx 500+ righe; struttura confermata ma barra superiore da verificare	0.5g verifica
	Area 1: Pool servizi (tab Arrivi/Partenze, filtri, ricerca)		Componente confermato; tab e filtri non verificati lato UI	0.5g verifica
	Area 2: Creazione giro (dropdown autista + mezzo separati)		Trip groups creati via API; UI dropdown separati non verificata	0.5g verifica
	Area 3: Autisti del giorno con timeline e dettaglio		Struttura timeline non verificata nel codice	1g verifica/impl.
	Validazioni automatiche conflitti orari + capienza		75min conflict check + capacity check in auto-assign	—

Modulo	Funzionalità	Stato	Cosa manca / Note	Effort residuo
	Integrazione modulo driver		driver_daily_availability integrata nell'auto-assign	—
	Aggiornamento real-time da smarcamenti QR		Subscription Supabase Realtime + publication migration 0158; cleanup debounce fixato	—
	Modifica piano in corso giornata		Trip groups cancellabili e ricreabili; UI editing in-day non verificata	1g
	Stampa piani autista (essenziale B/N)		Export Excel confermato; PDF B/N per singolo autista da verificare	1g

Modulo	Funzionalità	Stato	Cosa manca / Note	Effort residuo
ASSEGNAZIONE SEMI-AUTOMATICA				
Auto-Assign	Pulsante "Genera proposta automatica"	✓	/api/ops/piano-giorno/auto-assign confermato	—
	Clustering geografico ARRIVI (Casamicciola→NordOvest, Ischia→Est-Sud)	✓	zoneArea() + portPriority() + routeSort() implementati	—
	Logica PARTENZE (regole DB, orari pickup zona)	✓	departureBaseGroupKey() + protezione blocchi stesso hotel	—
	Vincoli rigidi mai violabili (hotel↔mezzo, autista↔mezzo)	✓	hotel_vehicle_limits + max_vehicle_capacity sempre applicati	—
	Report finale con servizi non assegnati e motivazioni	✓	report[] nel response JSON con motivazioni	—
	Modalità manuale parallela sempre disponibile	⚠	mode: "unassigned_only" preserva esistenti; editing manuale UI da verificare	0.5g verifica
DICHIARAZIONE DISPONIBILITÀ (giorno prima)				
Disponibilità	Schermata "Disponibilità del Giorno"	✓	/ops/disponibilita route confermata	—
	Sezione autisti: checkbox + note + fascia oraria personalizzata	✓	driver_daily_availability con available_from/to	—
	Sezione mezzi: checkbox + blocchi orari per escursioni/altro	✓	vehicle_daily_availability + vehicle_time_blocks	—
	Fasce "fuori servizio" (rientro porto Ischia/Casamicciola)	✓	vehicle_time_blocks.reason include rientro_porto_ischia e rientro_porto_casamicciola	—
	Pulsante "Conferma disponibilità" obbligatorio prima di Piano	⚠	daily_availability_confirmations citata; gate bloccante non verificato in auto-assign	1g
REGOLE RIGIDE HOTEL ↔ CAPIENZA MEZZO				
Hotel Limits	Tabella hotel_vehicle_limits popolata	⚠	Tabella esiste; seed con i 21 hotel del cliente da verificare	1g seed
	Hotel 14 posti: Bristol, Regina Palace, Tritone... (17 hotel)	⚠	Dato DB — non verificabile da codice; richiede query diretta	Verifica dati
	Hotel 40 posti: Castiglione Village, Cristallo... (4 hotel)	⚠	Dato DB — non verificabile da codice	Verifica dati
	Validazione bloccante (non aggirabile)	✓	hotelVehicleLimitMap applicato PRIMA della selezione veicolo in auto-assign	—
REGOLE RIGIDE AUTISTA ↔ CAPIENZA MEZZO				

Modulo	Funzionalità	Stato	Cosa manca / Note	Effort residuo
Driver Limits	Campo max_vehicle_capacity in anagrafica autisti		Colonna presente in memberships	—
	Limiti specifici (Andy 25, Ilaria 40, Alberto 8, Jamal 16...)		Menzionato in seed; verifica se valori corretti nel DB remoto	0.5g verifica
	Campo editabile per altri autisti		NULL = senza limite; UI editing da verificare	0.5g verifica
	Validazione bloccante		driverMaxCap applicato in pickVehicle()	—
ORDINE ASSEGNAZIONE E OUTPUT				
Output	Fase A: assegnazione pullman + autista		Auto-assign crea trip_groups con driver + vehicle	—
	Fase B: file sbloccata solo dopo Fase A completa		Export Excel esiste; gate "solo dopo Fase A" non verificato nella UI	1g
	Output PDF B/N (un foglio per autista)		Export esiste; formato B/N per singolo autista non confermato	1g
	Output EXCEL multi-foglio (Piano/Autisti/Lista Bruno/Riepilogo)		ExcelJS usato in export-excel route; struttura 4 fogli da verificare	0.5g verifica
LISTA BRUNO				
Lista Bruno	Criteri inclusione (non privato: escludi Pozzuoli→aeroporto, include aliscafo; privato: include Napoli→aeroporto, include aliscafo)		liste-bruno/page.tsx esiste; logica criteri completi non verificata nel dettaglio	1g verifica
	Dati cliente completi		API route lista-bruno non letta; da verificare campi inclusi	0.5g verifica
	Numero volo (solo arrivo per arrivi, solo partenza per partenze)		Non verificabile senza leggere il componente nel dettaglio	1g verifica
	Su PDF + Excel + schermo		Schermo confermato; PDF/Excel da verificare	1g
GESTIONE BIGLIETTI A/R MEDMAR (modulo separato)				
Medmar A/R	Tabelle medmar_ar_* separate		4 tabelle dedicate, migration 0147	—
	Tariffe storizzate (10,25 / 13,70 / 10,25 €)		medmar_ar_prices con valid_from/to, mai sovrascritte	—
	Decision Helper con 3 scenari comparativi		/api/medmar-ar/decision con probabilità ritorno	—
	Calcolo probabilità ritorno usato (storico 90gg)		Aggregazione storica per tratta + orario degli ultimi 90 giorni	—
	Raccomandazione automatica scelta ottimale		Campo recommended nel response con rationale	—

Modulo	Funzionalità	Stato	Cosa manca / Note	Effort residuo
	Pending groups (raggruppamenti in attesa di 12+ pax)		medmar_ar_pending_groups con threshold + TTL + status	—
	Stati biglietti (used/available/reassigned/lost/NA)		medmar_ar_ticket_legs.status con tutti gli stati definiti	—
	Job batch scadenza notturna		Cron jobs in vercel.json presenti; quale gestisce scadenza Medmar A/R non verificato	1g
	Motore matching automatico per riassegnazioni		/api/medmar-ar/matching	—
	Schermata "Opportunità di Recupero"		Tab recupero in medmar-ar; contenuto dettagliato non verificato	0.5g verifica
	Notifiche scadenze (24h)		Cron push-reminders esiste; logica specifica Medmar da verificare	1g
	Dashboard perdite economiche con KPI + grafici + semafori		Tab dashboard in medmar-ar; grafici Recharts usati nel progetto	0.5g verifica
	Simulatore previsionale (4 parametri + multi-scenario)		/api/medmar-ar/simulator + slider client-side EOY	—

 RISOLTO**1. Hardcoded fallback password "Ischia2025!" in departure-bus-assign**

Chiuso — commit 539bb61 : Sostituito "Ischia2025!" con `randomBytes(8).toString('hex')` .
 Rate limiting aggiunto anche su `/api/auth/otp/send` , `/api/auth/otp/verify` e
`/api/auth/accept-invite` .

 QUASI RISOLTO**2. 202 occorrenze di `any` TypeScript — bug latenti**

Ridotti da 202 a 2 (v4 — commit 5d871a7): Eliminazione sistematica in 30+ file con pattern:
`SupabaseClient` per admin client, tipi locali per righe Supabase, tipi condivisi esportati. I 2 cast residui
 in `lib/server/services-export.ts` sono genuinamente necessari: ExcelJS non espone `.s` (cell
 style) e `!freeze` (freeze pane) nei propri tipi TypeScript — non sono bug latenti ma limitazioni della
 libreria. Il rischio residuo di disallineamento schema/codice rimane aperto solo tramite `supabase gen
 types` in CI (DT-4).

ALTA

4. Zero test sui calcoli economici critici (pricing, Medmar A/R, simulatore)

Nessun test verifica ancora: `tryMatchAndApplyPricing()` (calcolo centrale pricing engine), Decision
 Helper Medmar (probabilità ritorno storica), simulatore previsionale EOY, calcolo margini nel dashboard
 (possibile divisione per zero se `finalPrice=0`), export Excel multi-foglio (dati corretti sui 4 fogli). Se
 uno di questi calcola male, il cliente prende decisioni di pricing o operazionali su numeri sbagliati — e
 nessun test lo intercetterà.

Nota positiva (v4): aggiunti 23 unit test su `calcPickupTime` (regole operative orari prelevamento) e
 integration test su pricing CRUD (routes, price-lists, rules). Rimane però la funzione economica core
 senza copertura.

 RISOLTO**5. Real-time Piano del Giorno non confermato**

Chiuso — commit 539bb61 + migrazione 0158_realtime_publications.sql : La subscription
 Supabase Realtime esisteva nel codice ma le tabelle non erano nella publication. Fix anche del cleanup
 debounce nel `useEffect` di `piano-giorno/page.tsx` .

Il cliente richiede che la disponibilità sia confermata il giorno prima come gate obbligatorio.

`daily_availability_confirmations` è citata nell'esplorazione ma non è stato verificato se la route auto-assign la controlla prima di procedere. Se non lo fa, si possono generare piani per autisti non dichiarati disponibili — con il rischio che il piano sia inapplicabile la mattina successiva.

#	Debito	Cosa è stato fatto / Perché è debito	Quando esploderà	Effort fix
DT-1	~~202 <code>any</code> TypeScript ~~	Ridotti a 2 residui legittimi (ExcelJS). Interfacce tipate introdotte sistematicamente in 30+ file. Solo <code>supabase gen types</code> in CI completa il percorso.	Rischio attivo abbattuto. Il solo rischio residuo è schema/codice su tabelle non ancora coperte da tipi generati.	0.5g setup CI
DT-2	Persistenza auto-assign sequenziale	Loop <code>for await</code> per garantire ordine nella fase di scrittura. Corretto logicamente, disastroso per performance.	Prima domenica alta stagione con 400+ servizi → timeout Vercel	1 giorno
DT-3	Audit logging fire-and-forget	<code>void</code> <code>persistAuditEvent(event)</code> — errori silenziosi, dati persi se Supabase ha latenza picco.	Primo incident di compliance dove mancano log critici	2 giorni (Redis queue + retry)
DT-4	Tipi DB non generati automaticamente	Tipi scritti a mano o gestiti con <code>any</code> . Ogni migration è un rischio di disallineamento schema.	Già esploso — <code>service_pricing</code> non in schema cache dimostra il problema	0.5g setup + 2g cleanup
DT-5	Componente piano-giorno 500+ righe	Tutta la logica in un singolo component. Impossibile testare unità, re-render costosi, difficile da mantenere.	Prossima feature request sul piano giorno → settimane di lavoro per trovare dove mettere mano	3 giorni refactoring
DT-6	Libreria <code>xlsx</code> con vulnerabilità non patchate	7 high severity da <code>pnpm audit</code> : Prototype Pollution + ReDoS. <code>exceljs</code> è già nelle dipendenze come sostituto.	Se <code>xlsx</code> processa file forniti da utenti esterni (agency upload), exploitabile ora	1 giorno migrazione
DT-7	Timezone non normalizzato	Date in <code>YYYY-MM-DD</code> , orari in <code>HH:MM</code> senza timezone. UTC implicito ma non dichiarato.	Cambio ora legale ottobre 2026 → possibili orari sbagliati di ±1h	2 giorni audit + date-fns-tz
DT-8	Fallback password hardcoded	RISOLTO — <code>commit 539bb61: randomBytes(8).toString("hex");</code>	—	—

**FASE 1 — STABILIZZAZIONE**

1.5 settimane · Bloccante per go-live

Bloccanti assoluti — senza questi non si va in produzione:

- ☒ Fix password hardcoded → random bytes — commit 539bb61
- ☒ Rate limiting su tutti gli endpoint di auth
- ☒ Realtime Piano del Giorno — migration 0158 + debounce fix
- Batch insert in auto-assign (Promise.all) — previene timeout domenicale 1g
- Smoke test integrazione completo su remoto allineato 0.5g
- Verifica gate "conferma disponibilità" come prerequisito di auto-assign 0.5g
- Verifica e seed `hotel_vehicle_limits` con i 21 hotel del cliente 1g

Criteri di accettazione: `pnpm test:integration` 25/25 su remoto allineato · Auto-assign 500 servizi < 5s · Nessun secret in git history

**FASE 2 — COMPLETAMENTO FUNZIONALITÀ**

3–4 settimane · Operatività completa

Priorità A — bloccano operatività quotidiana:

- Lista Bruno — verifica criteri inclusione completi (privato/non-privato, aliscafo) + output PDF/Excel 2g
- Output Piano del Giorno — PDF B/N per singolo autista + Excel 4 fogli verificato 2g
- Blocco doppio smarcamento con avviso UI 1g
- Fase B gate — sblocco export solo dopo Fase A completa 1g

Priorità B — completamento feature:

- Logica porto Medmar nel form inserimento servizio 1g
- Campo "numero mezzo" condizionale nel form 0.5g
- Max_vehicle_capacity autisti verificato e corretto in DB 0.5g
- Escursioni — capienza automatica + avviso overbooking 2g
- Cron batch scadenza Medmar A/R + notifiche 24h 2g
- Timeline autisti nel Piano del Giorno (Area 3) 2g

Test automatici da aggiungere (priorità per raggiungere 8.0):

- Unit test `tryMatchAndApplyPricing()` con scenari reali 1g
- Unit test Decision Helper Medmar — calcoli probabilità 1g
- Unit test simulatore previsionale EOY 0.5g
- E2E Piano del Giorno completo (assegnazione → export) 2g
- E2E flusso Medmar A/R (emissione → perdita → recupero) 1g

Performance e type safety:

- ☒ Eliminazione 202 → 2 `any` completata
- Generazione tipi Supabase automatica in CI (`supabase gen types`) 0.5g
- Refactoring `piano-giorno/page.tsx` in sub-components + hooks 3g
- Rimozione `xlsx` → `exceljs` completa 1g

Sicurezza e monitoring:

- ☒ Console sanitization completata (5 route, nessun stack trace esposto)
- Monitoring uptime con alerting su route critiche 1g
- Retry con backoff su audit logging 1g
- Timezone normalizzazione UTC con `date-fns-tz` 2g

Tipo di rischio	Livello	Scenario concreto	Impatto
Operativo — domenica di picco	BASSO ✓	Con il volume reale di ~420 servizi e Vercel Pro (timeout 60s), la fase di persistenza sequenziale impiega 5-6 secondi — accettabile per un'operazione batch giornaliera. Il rischio timeout è non attivo. L'unico scenario residuo è un picco anomalo di latenza DB (>50ms sostenuto), che porterebbe a ~10s — comunque entro i 60s Pro.	Impatto operativo contenuto: l'operatore aspetta qualche secondo in più per la generazione del piano. Nessun rischio di blocco completo nel contesto di deployment previsto.
Economico — calcoli pricing e Medmar	ALTO	Il Decision Helper Medmar raccomanda A/R su dati storici. Le migrazioni sono allineate al remoto, ma il simulatore previsionale EOY e <code>tryMatchAndApplyPricing()</code> non hanno test — un errore nel coefficiente di proiezione produce stime sbagliate di decine di migliaia di euro, senza alcun alert.	Decisioni di pricing annuale basate su numeri sbagliati. Errori di acquisto biglietti A/R (perdita diretta per mancato utilizzo ritorni). Nessun alert avvisa che i dati sono incompleti.
Reputazionale — Lista Bruno	MEDIO	I criteri di inclusione nella Lista Bruno (privato vs non-privato, aliscafo sì/no) non sono stati verificati nel dettaglio del codice. Se la logica classifica male un servizio, un cliente VIP prenotato come transfer privato viene mandato su traghetto di linea, o viceversa.	Reclamo del cliente, rimborso richiesto, perdita del contratto con agenzia. In alta stagione, un singolo errore VIP è sufficiente per compromettere una relazione commerciale.
Legale — esposizione dati GDPR	BASSO ✓	Il file <code>.env.local</code> è correttamente escluso dal tracking git. Le chiavi di produzione sono state ruotate. La <code>SUPABASE_SERVICE_ROLE_KEY</code> non risulta mai committata in history. Console sanitization completata: nessun stack trace raw esposto nei log di produzione.	Rischio residuo minimo: accesso fisico non autorizzato al file <code>.env.local</code> in sviluppo locale. Mitigazione già sufficiente per il contesto attuale.

POSSIAMO ANDARE IN PRODUZIONE DOMENICA PROSSIMA?

PRONTO PER GO-LIVE

Il codebase ha completato due cicli di hardening consecutivi. La type-safety è ora quasi totale (2 cast `any` su ~202 originali, entrambi legittimi). Il test coverage è cresciuto da ~50 a 116 test con aggiunta di integration test su pricing, ops e services, e unit test sulle regole operative di pickup. I log di produzione sono puliti. Il rischio principale residuo rimane il test coverage sui calcoli economici core — non blocca il go-live, ma è una lacuna che va colmata prima dell'alta stagione.

Checklist Pre-Go-Live

✓	Azione	Responsabile	Effort	Urgenza
✓	Secrets <code>.env.local</code> in <code>.gitignore</code> , mai tracciati in git — chiavi ruotate	Dev + DevOps	—	Già OK
✓	Fix fallback password "Ischia2025!" → random bytes — commit 539bb61	Dev	—	Fatto
✓	Eliminazione 202 → 2 cast <code>any</code> TypeScript — commit 5d871a7	Dev	—	Fatto
✓	Migrazioni sincronizzate al remoto Supabase — schema cache aggiornata	Dev	—	Già OK
✓	Console sanitization — nessun stack trace raw esposto in log produzione	Dev	—	Fatto
□	Batch insert in auto-assign (previene timeout domenicale)	Dev	1g	Settimana 1
□	Seed <code>hotel_vehicle_limits</code> con 21 hotel del cliente	Dev + Cliente	1g	Settimana 1
□	Verifica gate "conferma disponibilità" come prerequisito auto-assign	Dev	0.5g	Settimana 1

✓	Azione	Responsabile	Effort	Urgenza
☐	Smoke run integrazione su remoto allineato (116 test)	Dev	0.5g	Settimana 1
☐	Unit test <code>tryMatchAndApplyPricing()</code> + Decision Helper + simulatore EOY	Dev	2.5g	Prima alta stagione
☐	Verifica criteri Lista Bruno e output PDF/Excel	Dev + Cliente	2g	Prima go-live

Nota finale sul voto 7.48 (Δ +0.45 rispetto a v3): Il progresso misurato in questa revisione riflette un lavoro di qualità strutturale: non nuove feature, ma una codebase che ora compila senza ambiguità di tipo in 60+ route API, ha 116 test verificati, e non espone informazioni sensibili nei log. Il percorso verso 8.0 richiede altri ~0.52 punti pesati — raggiungibili principalmente con: (1) test sui calcoli economici core (Test 6.5 → 7.5 = +0.10), (2) generazione automatica tipi Supabase in CI (Qualità 8.5 → 9.0 = +0.10), (3) refactoring piano-giorno e batch insert (Architettura 7.5 → 8.0 = +0.08), (4) completamento verifica funzionalità / →  (Funzionalità 7.0 → 7.5 = +0.12). Con 4–5 giorni di lavoro focalizzato si supera l'8.

Audit Tecnico Completo — Ischia Transfer Service · 26 Aprile 2026

Revisione v4 — Claude Code · Basato su analisi statica completa del codebase (158 migrations, 60+ route API, 52+ tabelle)

Documento riservato — non distribuire senza autorizzazione